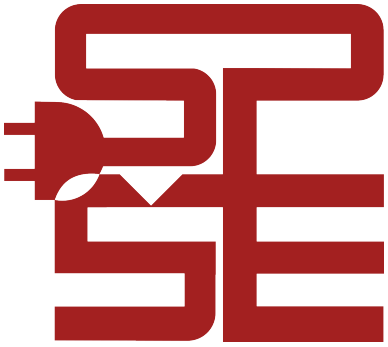


SPSE ARM kit

Register-level STM32 learning kit

SPSE Havirov

Version 4.0.0

SPSE ARM kit 1

Using the libraries . . . 1

Timing — `chrono.h` . . . 1

I2C — `I2C.h` . . . 1

LCD — `LCD_I2C.h` or `LCD.h` . . . 2

Keypad — `KPAD.h` . . . 2

Pin Assignments & Hardware 3

Identifying your board . . . 3

Kit versions . . . 3

Pin assignments . . . 4

Setting up a project in SEGGER Embedded Studio 5

Hardware . . . 5

Step 1 – Create a new project . . . 5

Step 2 – Select the project template and set the name . . . 5

Step 3 – Select the target processor . . . 6

Step 4 – Select files to add . . . 6

Step 5 – Select configurations . . . 7

Step 6 – Open project Options . . . 7

Step 7 – Set the debugger target connection . . . 8

Step 8 – Configure the GDB server . . . 9

Step 9 – Copy the include folder into the project directory . . . 9

Step 10 – Add include files and build . . . 10

Step 11 – Close the solution when done . . . 10

Switching between boards . . . 11

Notes . . . 11

Appendix: Schematics 12

SPSE ARM kit

Register-level STM32 learning kit for SPSE Havirov. The same SEGGER Embedded Studio project supports three board configurations.

Configuration	Board	LCD
F411 Debug / Release	NUCLEO-F411RE + SPSE shield	I2C
F407 Debug / Release	STM32F4-DISCOVERY	I2C or direct
G071 Debug / Release	NUCLEO-G071RB + SPSE shield	I2C

For project setup, see Getting started in SEGGER Embedded Studio. For the complete board-to-board pin comparison, see Pin Assignments & Hardware.

Using the libraries

Include `kit.h` first. It selects the STM32 device header and board definition from the active SES configuration. Initialize SysTick before calling `delay()` or any library function that uses it.

Timing — `chrono.h`

Function or setting	Parameters	Purpose
`SysTick_Config(reload)`	`reload`: `SystemCoreClock / CLK_DIV`	Configure SysTick. `CLK_DIV` defaults to 1000, giving a 1 ms tick.
`delay(value)`	`value`: number of ticks	Wait for the requested number of ticks.
`cekej(value)`	`value`: busy-loop count	Simple busy-loop delay that does not need SysTick.

I2C — `I2C.h`

The F-board I2C1 driver is used by the I2C LCD library. Call `I2C_GPIO_init()` and `I2C_init()` before initializing the LCD.

Function	Parameters	Purpose
`I2C_GPIO_init()`	—	Configure the I2C1 clock and data pins.
`I2C_init()`	—	Initialize I2C1 at 100 kHz.
`I2C_probe(addr)`	`addr`: shifted 8-bit address	Return 1 if a device acknowledges, otherwise 0.
`I2C_start()`	—	Wait for an idle bus / begin a transaction.
`I2C_send_addr(addr)`	`addr`: shifted 8-bit address	Select the target device.
`I2C_write(data)`	`data`: one byte	Send one byte.
`I2C_stop()`	—	Finish the transaction.

For STM32G071, `I2Cg.h` is the alternative to the normal F-board `I2C.h` driver.

LCD — `LCD_I2C.h` or `LCD.h`

Use `LCD_I2C.h` for an LCD connected through an I2C expander. Use `LCD.h` for the directly connected LCD on F407. The cursor interface is the same in both: `LCD_setCursor(row, col)`, with zero-based row and column.

Function	Parameters	Purpose
`LCD_init()` / `LCD_setup()`	—	Initialize the LCD. `LCD_init()` is for I2C; `LCD_setup()` is for direct connection.
`LCD_clear()`	—	Clear the display and return to the first character.
`LCD_setCursor(row, col)`	`row`, `col`: zero-based position	Move the cursor.
`LCD_put(data)`	`data`: character code	Display one character.
`LCD_print(text)`	`text`: zero-terminated string	Display a string. It does not wrap lines.
`LCD_set(cmd)`	`cmd`: LCD command	Send a display command.
`LCD_customChar_write()`	Slot 0–7; eight-byte bitmap	Store a custom character in CGRAM (direct LCD driver).
`LCD_customChar_read()`	Slot 0–7	Display the selected custom character (direct LCD driver).

For STM32G071, `LCD_I2Cg.h` is the alternative to the normal F-board `LCD_I2C.h` driver.

Keypad — `KPAD.h`

Define `KPAD_KeyMap[KPAD_ROWS][KPAD_COLS]` before including the driver. `KPAD_COLS` selects three or four columns; the default is four. `KPAD_STEP` sets the debounce delay in milliseconds and defaults to 250.

Function	Parameters	Purpose
`KPAD_setup()`	—	Configure keypad GPIO.
`KPAD_getKey()`	—	Return the mapped character for a pressed key, or 0 when no key is pressed.

For STM32G071, `KPADg.h` is the alternative to the normal F-board `KPAD.h` driver.

Pin Assignments & Hardware

Identifying your board

Flip the Nucleo board over. The board type is printed on the label in the centre. The two currently supported Nucleo variants are F411RE and G071RB.

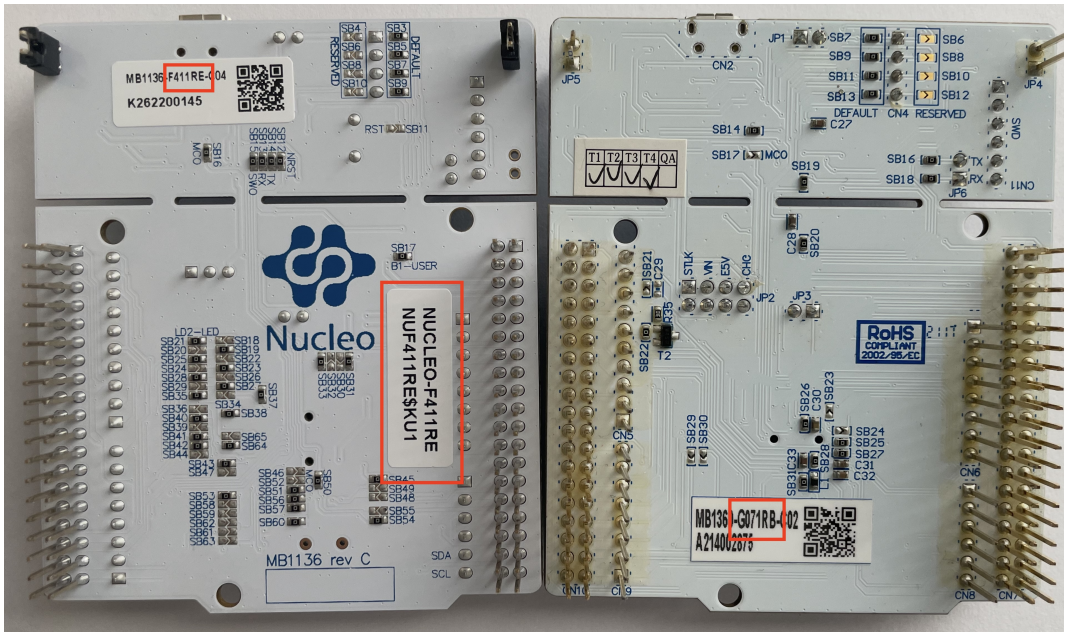
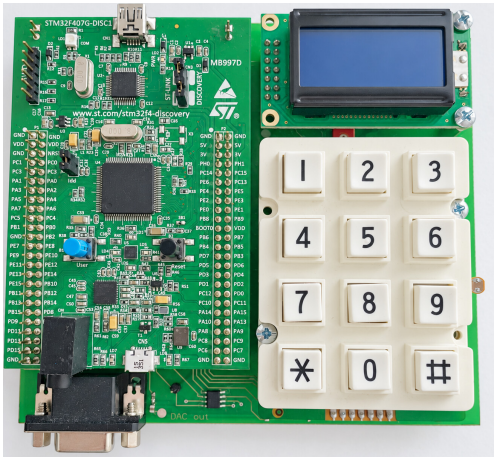


Figure 1 Back of the board -- F411RE (left) and G071RB (right)

Kit versions



New kit – NUCLEO + SPSE shield (2026)



Older kit – STM32F4-DISCOVERY

Pin assignments

Function	Arduino	NUCLEO-F411RE	NUCLEO-G071RB	F4-DISCOVERY
LED D1	A3	PB0	PB1	PE12
LED D2	D3	PB3	PB3	PE13
LED D3	D5	PB4	PB4	PE14
LED D4	D4	PB5	PB5	PE15
LED D5	D10	PB6	PB0	–
LED D6	D6	PB10	PB14	–
LED D7	A5	PC0	PB12	–
LED D8	A4	PC1	PB11	–
KPAD ROW0	A0	PA0	PA0	PD6
KPAD ROW1	A2	PA4	PA4	PD7
KPAD ROW2	D13	PA5 (+LD2)	PA5 (+LD4)	PD8
KPAD ROW3	D12	PA6	PA6	PD9
KPAD COL0	D11	PA7	PA7	PD0
KPAD COL1	D7	PA8	PA8 (*)	PD1
KPAD COL2	D8	PA9	PA9	PD2
KPAD COL3	D2	PA10	PA10	PD3
LCD I2C SDA	D14	PB9 (AF4)	PB9 (AF6)	PB7 (AF4)
LCD I2C SCL	D15	PB8 (AF4)	PB8 (AF6)	PB6 (AF4)
LCD direct RS	–	–	–	PE3
LCD direct R/W	–	–	–	PE4
LCD direct E	–	–	–	PE5
LCD direct DB4-DB7	–	–	–	PE6-PE9
Photoresistor	A1	PA1 (ADC1_IN1)	PA1 (ADC1_IN1)	–
Shield button S1	D9	PC7	PC7	–
On-board LED	–	PA5 (LD2)	PA5 (LD4)	PD12-PD15
User button B1	–	PC13 (press=0)	PC13 (press=0)	PA0 (press=1)
UART TX/RX	–	PA2/PA3 USART2 AF7	PA2/PA3 USART2 AF1	PA2/PA3 USART2 AF7

(*) G071: PA8 is also UCPD1 CC1 – has a 5.1 kOhm pull-down after reset. `KPAD_setup()` disables it via `UCPD1_STROBE` in `SYSCFG->CFGR1`. */

Setting up a project in SEGGER Embedded Studio

Hardware

The kit consists of a Nucleo-64 development board and the SPSE Expansion Board shield.
The older version of the kit uses an STM32F4-DISCOVERY board instead of a Nucleo.

Step 1 – Create a new project

On the dashboard click **Create new**.

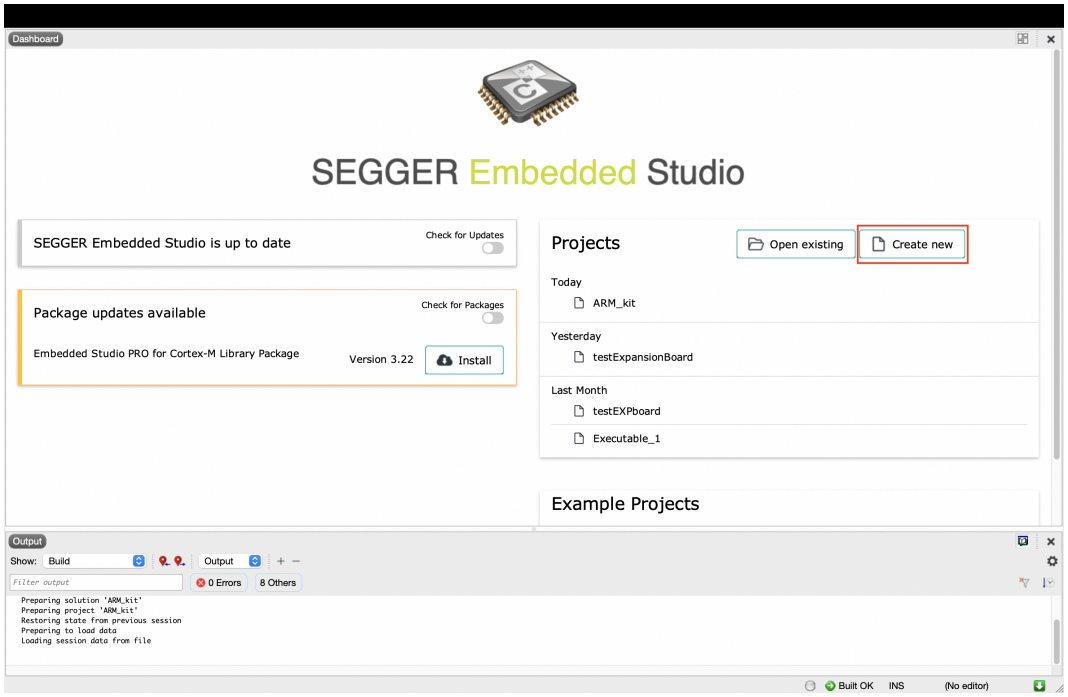


Figure 2 Dashboard -- click Create new

Step 2 – Select the project template and set the name

Select **A C/C++ executable for STMicroelectronics STM32F4xx** from the list. Enter the project **Name** and choose a **Location** on disk, then click **Next**.

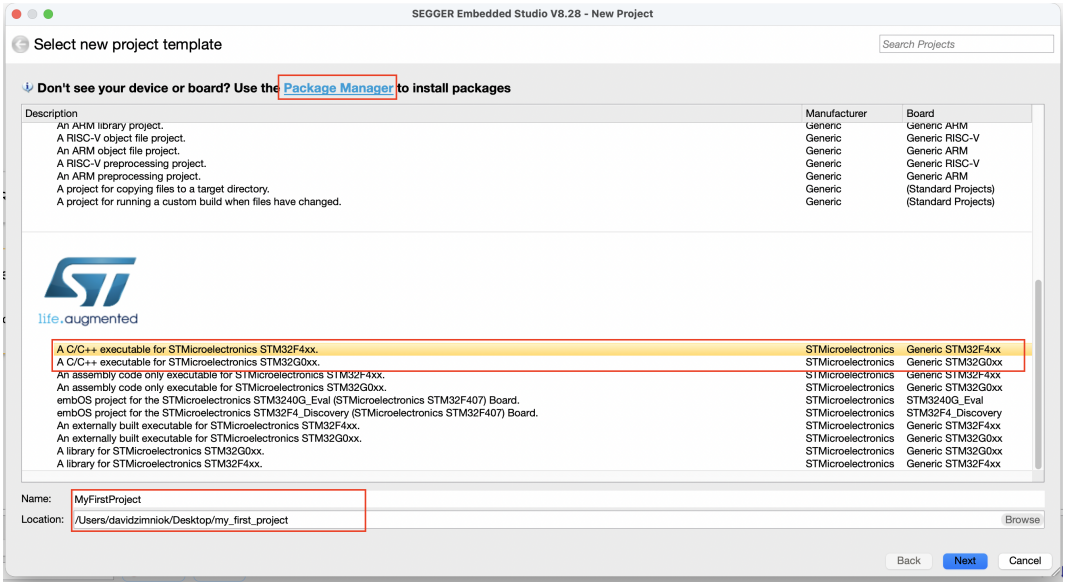


Figure 3 Select template, enter project name and location

Step 3 – Select the target processor

Click on **Target Processor** and select **STM32F411RETx** from the dropdown, then click **Next**.

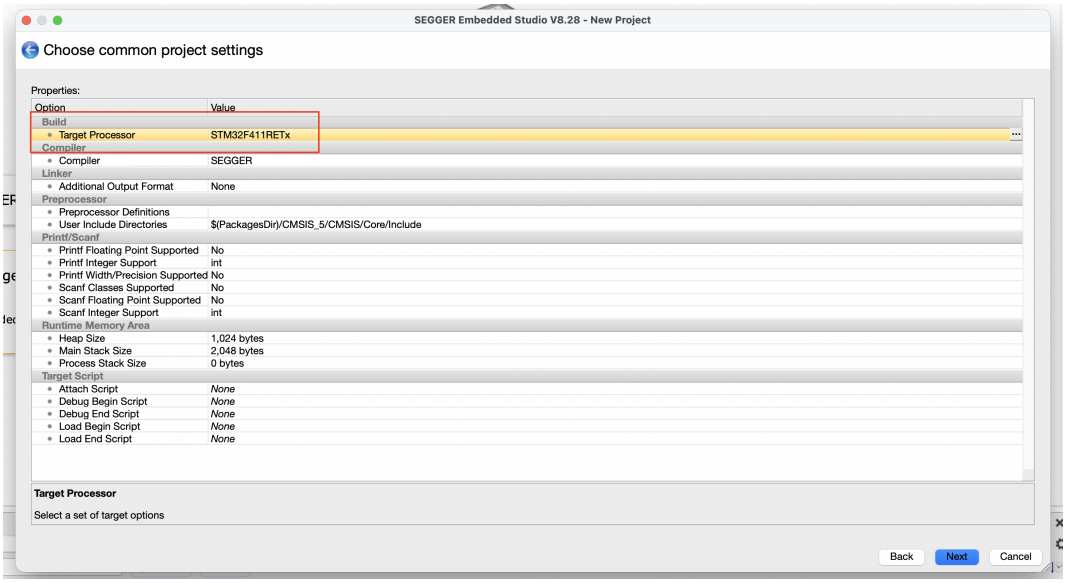


Figure 4 Select Target Processor: STM32F411RETx

Step 4 – Select files to add

Leave all checkboxes as they are and click **Next**.

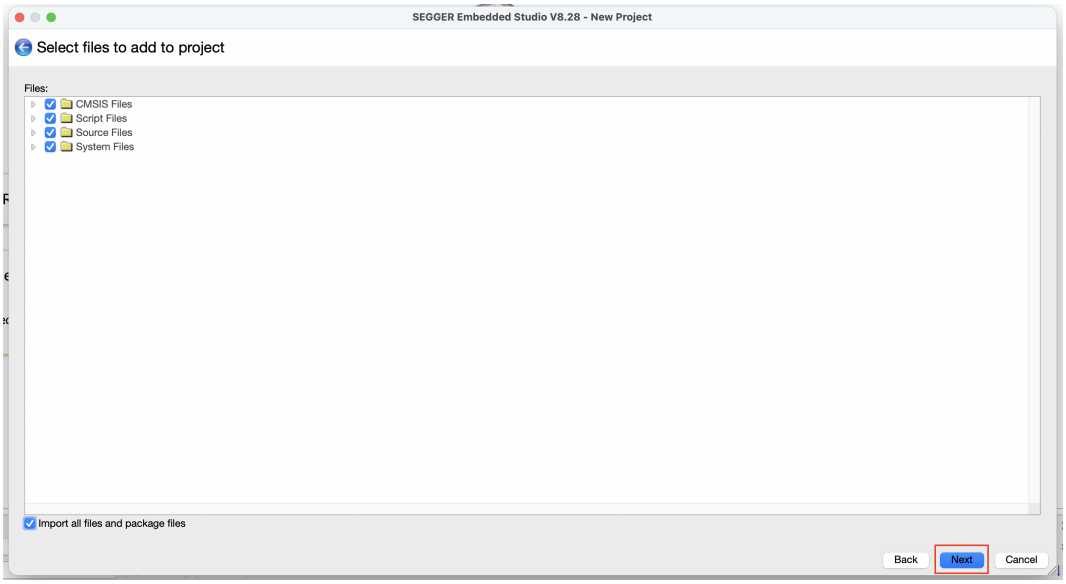


Figure 5 Select files -- leave defaults and click Next

Step 5 – Select configurations

Leave **Debug** and **Release** both checked and click **Finish**.

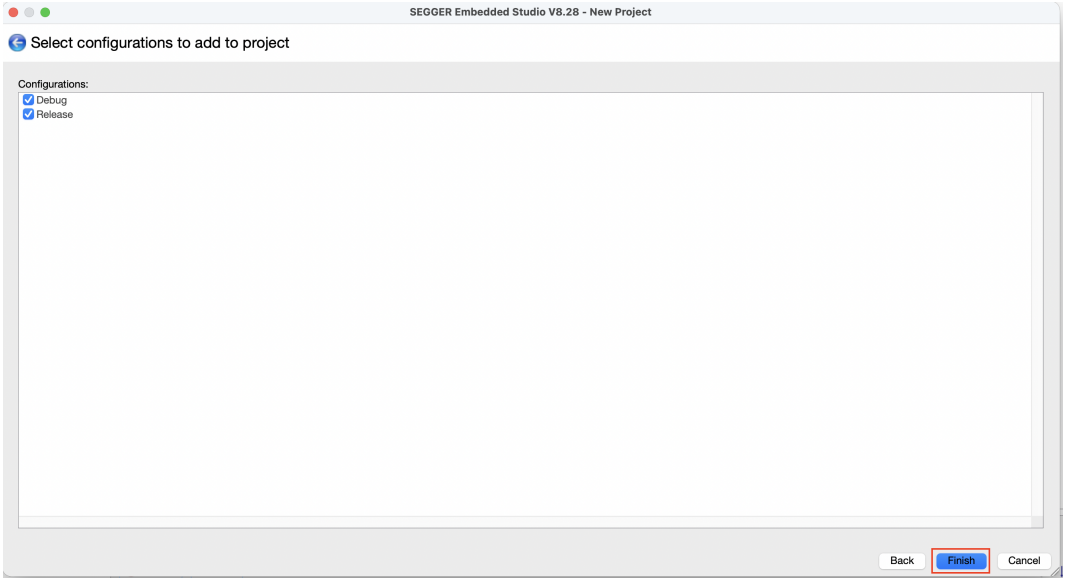


Figure 6 Select configurations -- click Finish

Step 6 – Open project Options

In the *Project Explorer* on the left, right-click the **project name** (not the Solution above it) and choose **Options**.

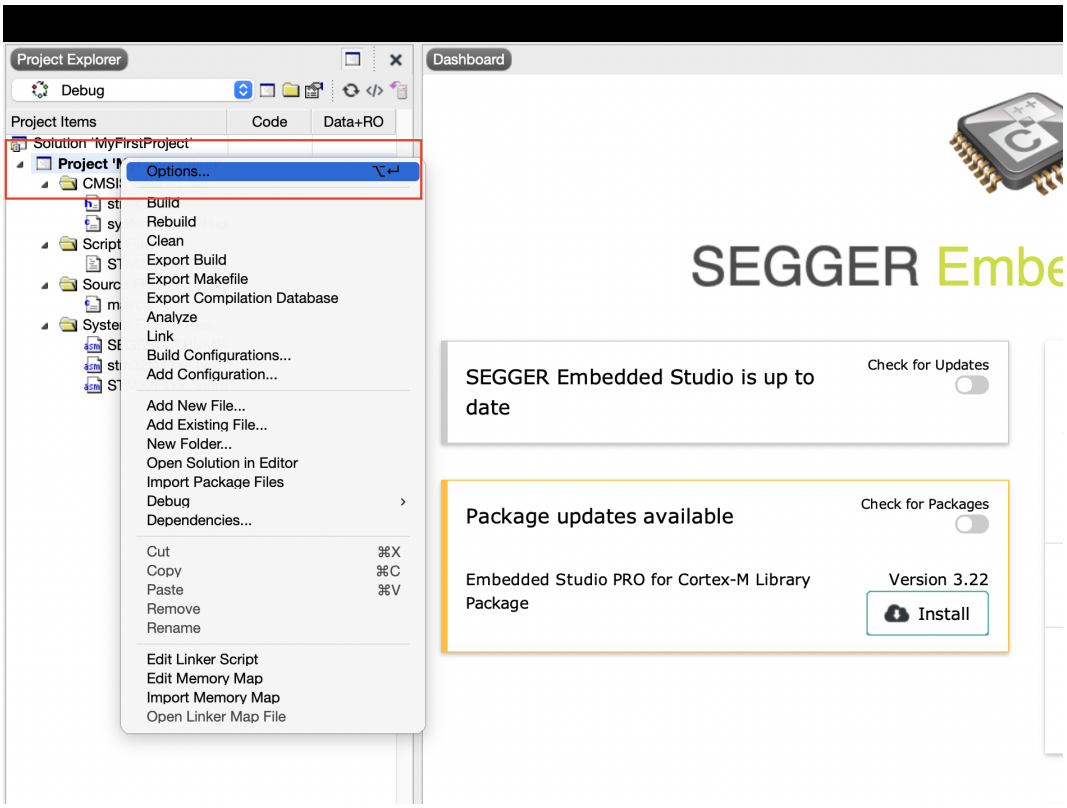


Figure 7 Right-click the project name and choose Options

Step 7 – Set the debugger target connection

In the Options window, make sure configuration is set to **Common**. Navigate to **Debug ->** **Debugger** and set **Target Connection** to **GDB Server**.

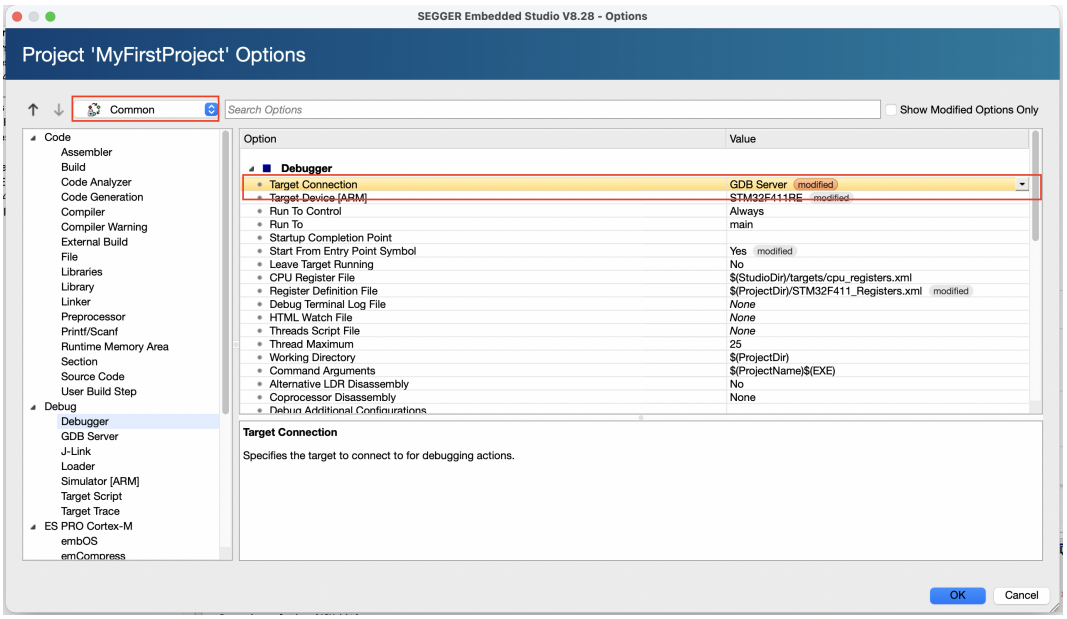


Figure 8 Debugger -- set Target Connection to GDB Server

Step 8 – Configure the GDB server

Navigate to **Debug ->** **GDB Server** and fill in the following values:

Option	Value
Type	ST-LINK
GDB Server Command Line	(see below)
Auto Start GDB Server	Yes
Port	61234

GDB Server Command Line:

```
/Library/STMicroelectronics/STM32Tools/gdbserver/bin/ST-LINK_gdbserver -d -cp
      "/Library/STMicroelectronics/STM32Tools/cubeprogrammer/bin" -e -r 10
```

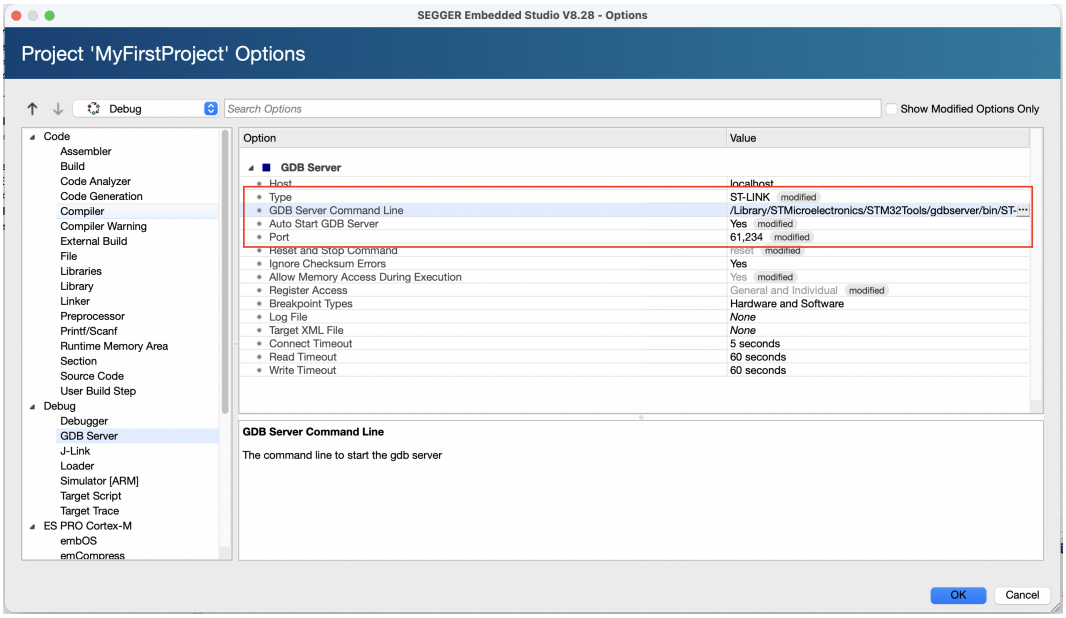


Figure 9 GDB Server settings

Click **OK** to save the project options.

Step 9 – Copy the include folder into the project directory

Open the project folder in Finder. Copy the **include** folder from the ARM kit repository into the project folder (next to `main.c`).

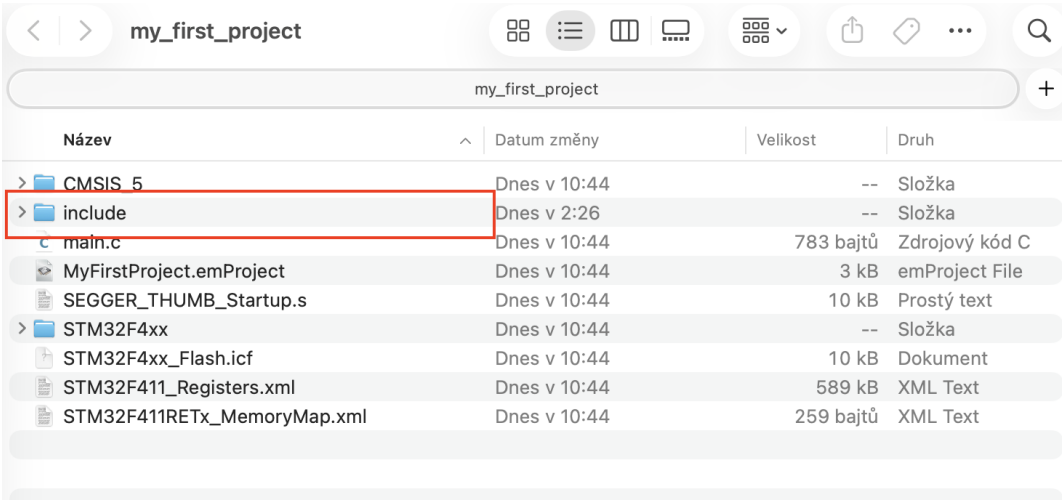


Figure 10 Copy the include folder into the project directory

Step 10 – Add include files and build

Back in SES, right-click **Source Files** and choose **Add Existing File**. Navigate into `include` and add the files you need. Press **F7** to build – the Output panel should show **0 Errors**.

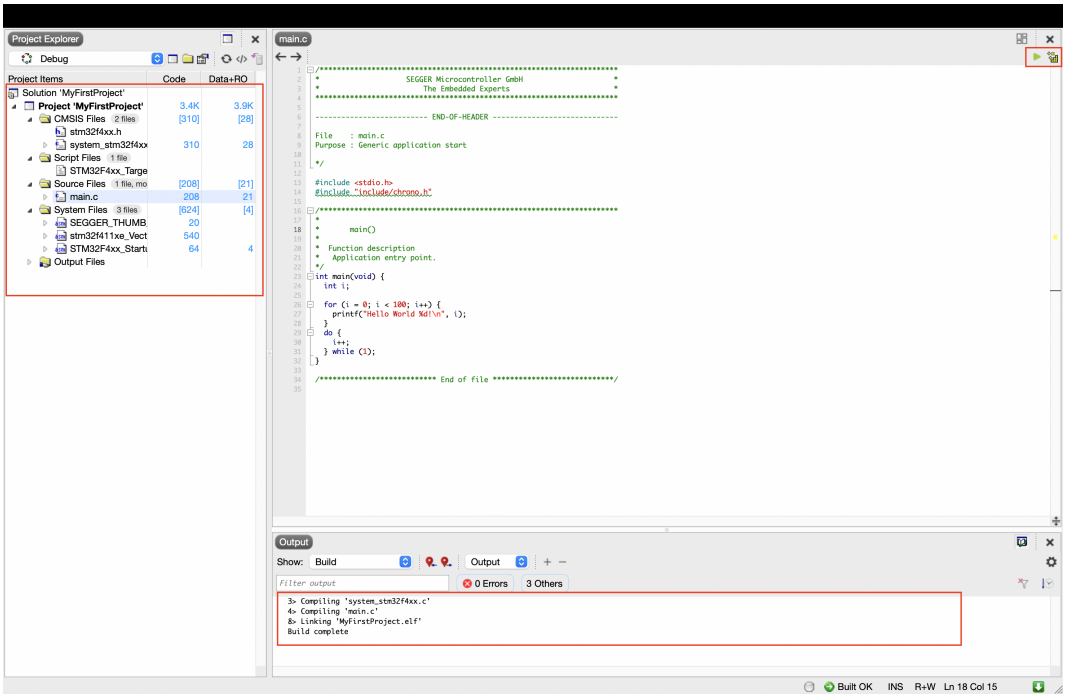


Figure 11 Project after successful build

Step 11 – Close the solution when done

When you finish working, close via **File ->** **Close Solution**.

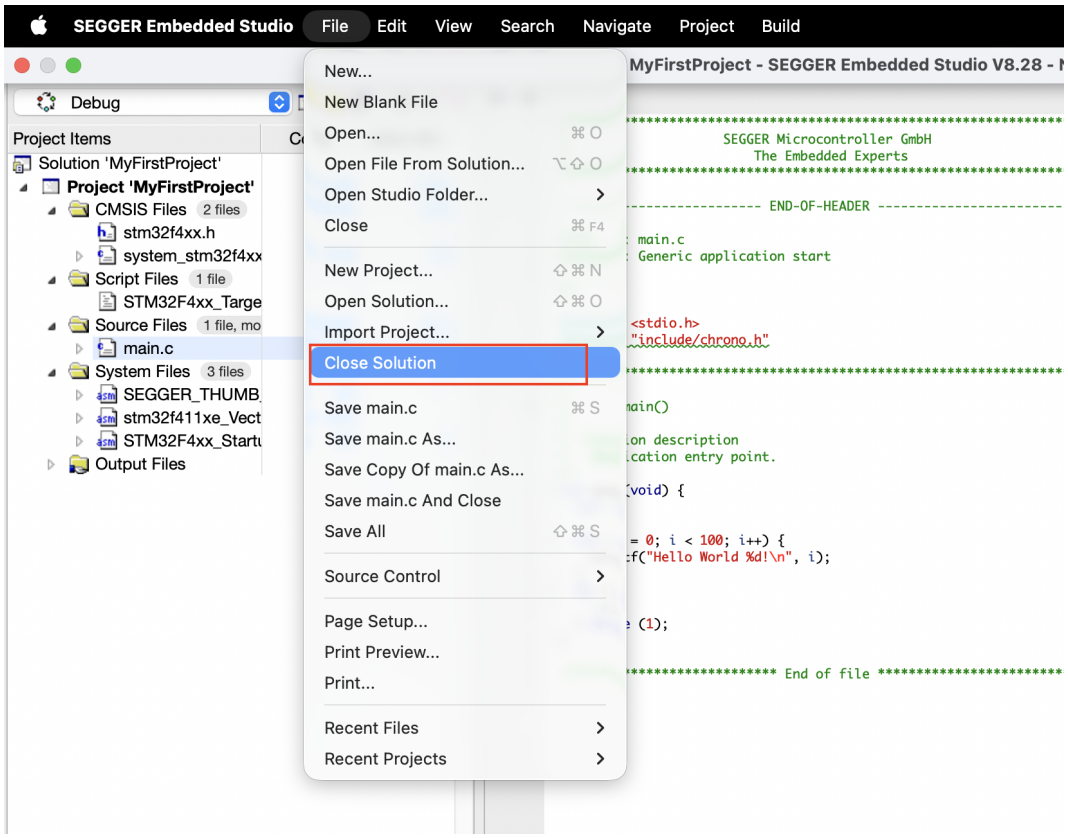


Figure 12 File menu -- Close Solution

Switching between boards

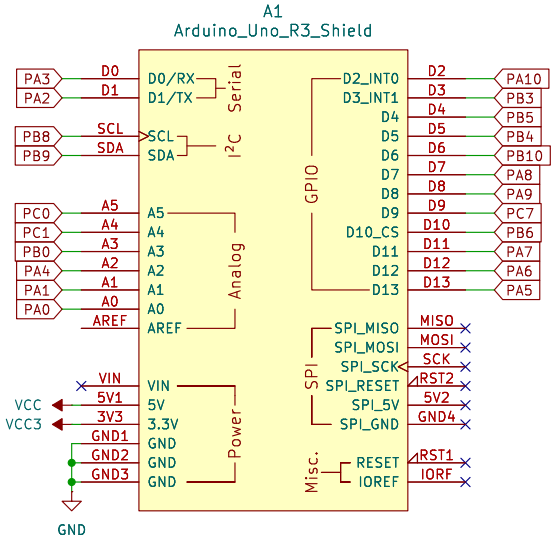
The ARM kit project supports all three boards in a single project file. Select the board from the configuration drop-down in the top-left toolbar, using the board table at the start of this manual to choose the matching configuration.

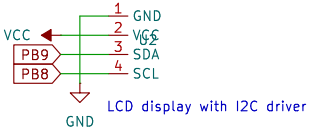


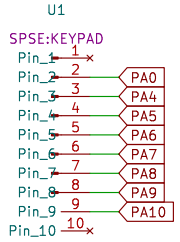
Figure 13 Configuration dropdown showing F407 / F411 / G071

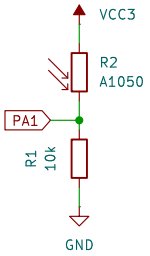
Notes

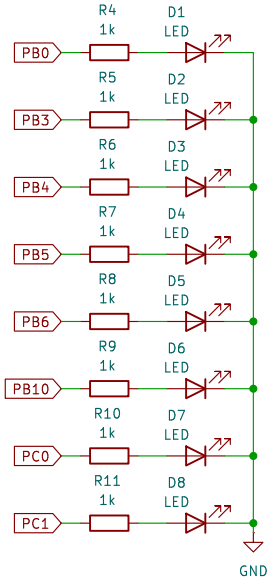
- Only **one file with `main()`** may be compiled at a time. To switch to an example: right-click it -> *Options* -> *Exclude From Build* = **No**, and set `src/main.c` -> *Exclude From Build* = **Yes**. Reverse to switch back.
- **F7** = build, **F5** = flash and debug.
- If the build fails with "No such file or directory" for a kit header, check that *Code* -> *Preprocessor* -> *User Include Directories* contains `$(ProjectDir)/include.*/`

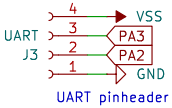


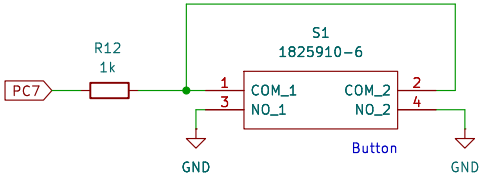












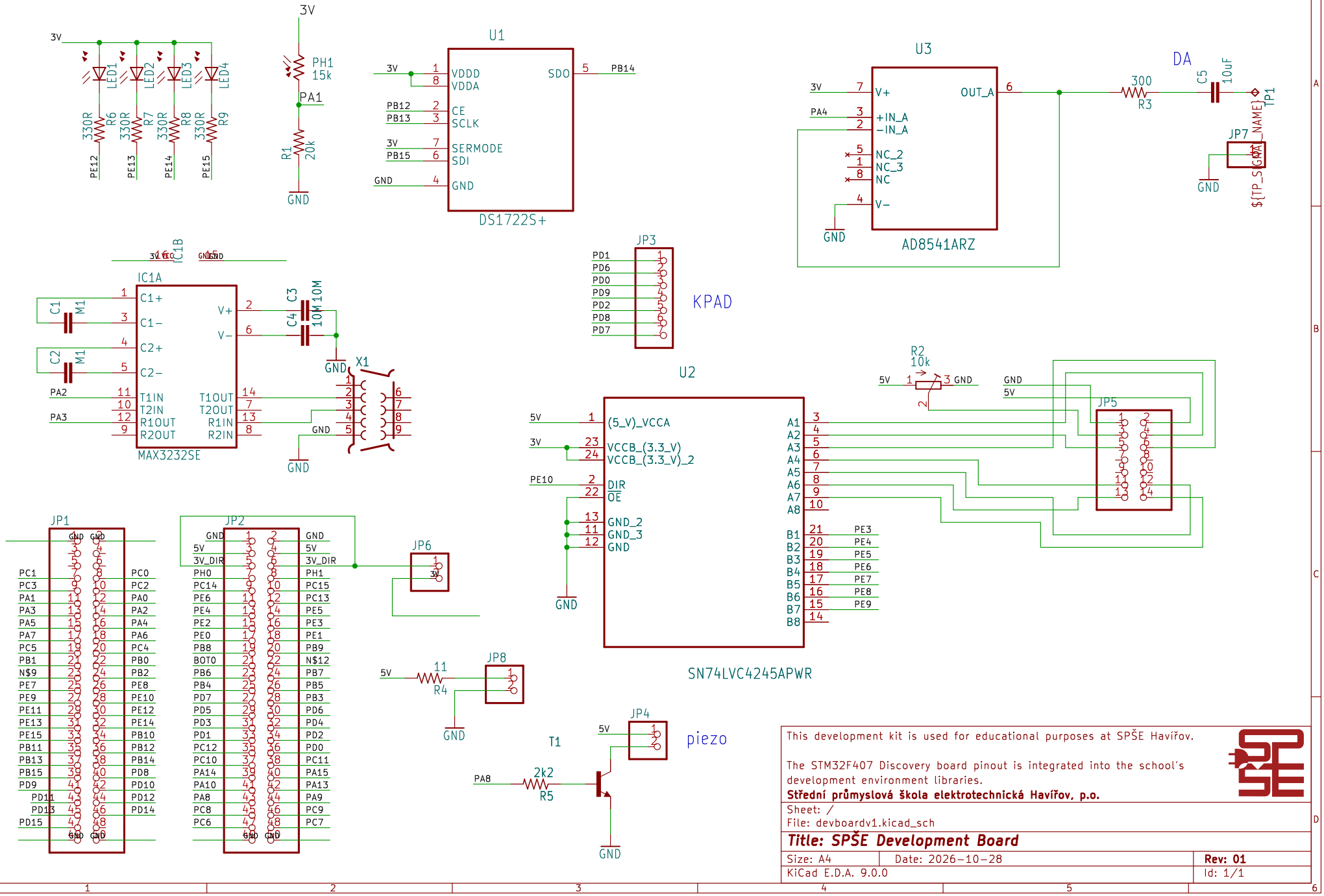
This development board is designed for educational purposes at SPŠE Havířov.

The board uses the Arduino shield layout.
This pinout mapping is designed for the NUCLEO-F411RE development board.
Střední průmyslová škola elektrotechnická Havířov, p.o.

Sheet: /
File: SPSE_deska.kicad_sch

Title: SPŠE Development Board (2026)

Size: A4	Date: 2026-10-28	**Rev: 03**
KiCad E.D.A. 9.0.0		Id: 1/1


3V
LED1
LED2
LED3
LED4
330R
R6
R7
R8
R9
PE12
PE13
PE14
PE15
PH1
15k
PA1
R1
20k
GND
U1
VDDD
VDDA
CE
SCLK
SERMODE
SDI
GND
SDO
PB14
PB12
PB13
PB15
DS1722S+
U3
V+
+IN_A
−IN_A
NC_2
NC_3
NC
V−
OUT_A
AD8541ARZ
PA4
DA
300
R3
C5
10uF
TP1
JP7
${TP_SIGNAL_NAME}
IC1A
IC1B
C1+
C1−
C2+
C2−
V+
V−
T1IN
T2IN
R1OUT
R2OUT
T1OUT
T2OUT
R1IN
R2IN
MAX3232SE
C1
C2
C3
C4
M1
10M
X1
PA2
PA3
JP3
KPAD
PD1
PD6
PD0
PD9
PD2
PD8
PD7
U2
(5_V)_VCCA
VCCB_(3.3_V)
VCCB_(3.3_V)_2
DIR
OE
GND_2
GND_3
SN74LVC4245APWR
PE10
R2
10k
JP5
JP1
JP2
JP6
JP8
R4
11
JP4
piezo
T1
2k2
R5
PA8
PC1
PC3
PA1
PA3
PA5
PA7
PC5
PB1
N$9
PE7
PE9
PE11
PE13
PE15
PB11
PB13
PB15
PD9
PD11
PD13
PD15
PC0
PC2
PA0
PA2
PA4
PA6
PC4
PB0
PB2
PE8
PE10
PE12
PE14
PB10
PB12
PB14
PD8
PD10
PD12
PD14
PH0
PC14
PE6
PE4
PE2
PE0
PB8
BOT0
PB6
PB4
PD7
PD5
PD3
PD1
PC12
PC10
PA14
PA10
PA8
PC8
PC6
PH1
PC15
PC13
PE5
PE3
PE1
PB9
N$12
PB7
PB5
PB3
PD6
PD4
PD2
PD0
PC11
PA15
PA13
PA9
PC9
PC7
PE3
PE4
PE5
PE6
PE7
PE8
PE9
This development kit is used for educational purposes at SPŠE Havířov.
The STM32F407 Discovery board pinout is integrated into the school's development environment libraries.
Střední průmyslová škola elektrotechnická Havířov, p.o.
Sheet: /
File: devboardv1.kicad_sch
Title: SPŠE Development Board
Size: A4
Date: 2026-10-28
Rev: 01
KiCad E.D.A. 9.0.0
Id: 1/1